AI ChatBot

The ChatBot is a Python application that allows you to chat with multiple PDF documents. You can ask questions about the PDFs using natural language, and the application will provide relevant responses based on the content of the documents. This app utilizes a language model to generate accurate answers to your queries. Please note that the app will only respond to questions related to the loaded PDFs.

This is a Fine Tunning based application

Restricted | © Siemens 2024 | Siemens Digital Industries Software

How It Works

The application follows these steps to provide responses to your questions:

1. PDF Loading: The app reads multiple PDF documents and extracts their text content (Limit 200 Mb per File)

2. Text Chunking: The extracted text is divided into smaller chunks that can be processed effectively.

3. Language Model: The application utilizes a language model to generate vector representations (embeddings) of the text chunks.

- Embeddings can be stored in a database (DB)
- You can select in which DB to store your data
- Can Select the Manuals you wish to retrieve from the already analyzed PDFs

4. Similarity Matching: When you ask a question, the app compares it with the text chunks and identifies the most semantically similar ones.

5. Response Generation: The selected chunks are passed to the language model, which generates a response based on the relevant content of the PDFs.

Restricted | © Siemens 2024 | Siemens Digital Industries Software

Chunking with Unstructured

The extracted text is divided into smaller chunks that can be processed effectively.
Chunking has 2 processes : Document Elements Structure Detection, Strategy of Splitting the Chunks

Document Elements Structure Detection

- The chunks are done taking in consideration the Manuals structure
  Document Elements are detected in base of the PDF Manual page Structure

- Unstructured uses specific knowledge about each document format to partition the document into semantic units (Document Elements), we only need to resort to text-splitting when a single element exceeds the desired maximum chunk size. Except in that case, all chunks contain one or more whole elements, preserving the coherence of semantic units established during partitioning.

- In general, chunking combines consecutive elements to form chunks as large as possible without exceeding the maximum chunk size.
  A single element that by itself exceeds the maximum chunk size is divided into two or more chunks using text-splitting.

- Elements Detection produces a sequence of :
  CompositeElement {Image, NarrativeText, Title, …..}, Table, TableChunk elements.
  Each “chunk” is an instance of one of these 3 types.

Restricted | © Siemens 2024 | Siemens Digital Industries Software

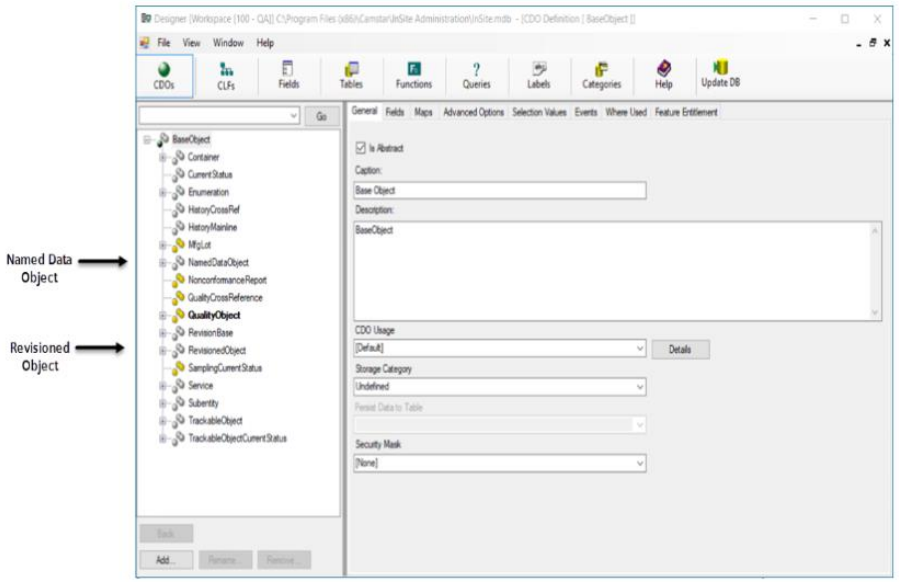
Modeling in Designer

CompositeElement: Title

Designer is the graphical application provided with Opcenter EX MDD and Opcenter EX CR for managing the CDOs. As stated previously, you define and maintain the modeling objects, or elements, of your factory information model. Modeling objects, represented by NamedDataObjects (NDOs) and RevisionedObjects (ROs), are a subset of the configurable data objects (CDOs) in Designer.

- A Named Data Object is a class of objects within the application identified by a unique name.
- A Revisioned Object is a class of objects within the application identified by a unique name and a revision. Instances of data objects are persistent; that is, information about them is written to the database.

This image shows the Designer CDO Definition page.



CompositeElement: Image

Instances of the CDOs are created and maintained in Modeling, while the objects themselves are created (and can be renamed) in Designer. For example, a factory administrator creates two new Employee definitions, Operator and Supervisor, in the application. An administrator must use Designer to rename one of the Employee objects as Personnel.

Names of objects are displayed with spaces in Modeling; for example, ObjectGroup in Designer becomes Object Group in Modeling. This is done through the object's Display Name attribute.

You can modify the properties of an object field in Designer. For example, the Customer field, which is optional in the Product definition, can be designated as required by checking the appropriate box in the Processing Options of the field properties.

CompositeElement: Narrative Text

This are the Elements types that will be Detected by:

Elements Structure Detection

Restricted | ©

Strategy: «By Title»

Once we have detected the structure (Document Elements) of the text,
we now group them, in order to create the Chunks
The strategy used to create the chunks is "by title"

By Title Chunking Strategy:

- Preserves section boundaries and optionally page boundaries.
- Detects section headings and starts a new chunk when a Title element is encountered.
- Can respect page boundaries to keep elements on different pages in separate chunks.
- Combines small sections to fill chunks, adjustable with combine_text_under_n_chars

Restricted | © Ergi Toshkezi

Embedding

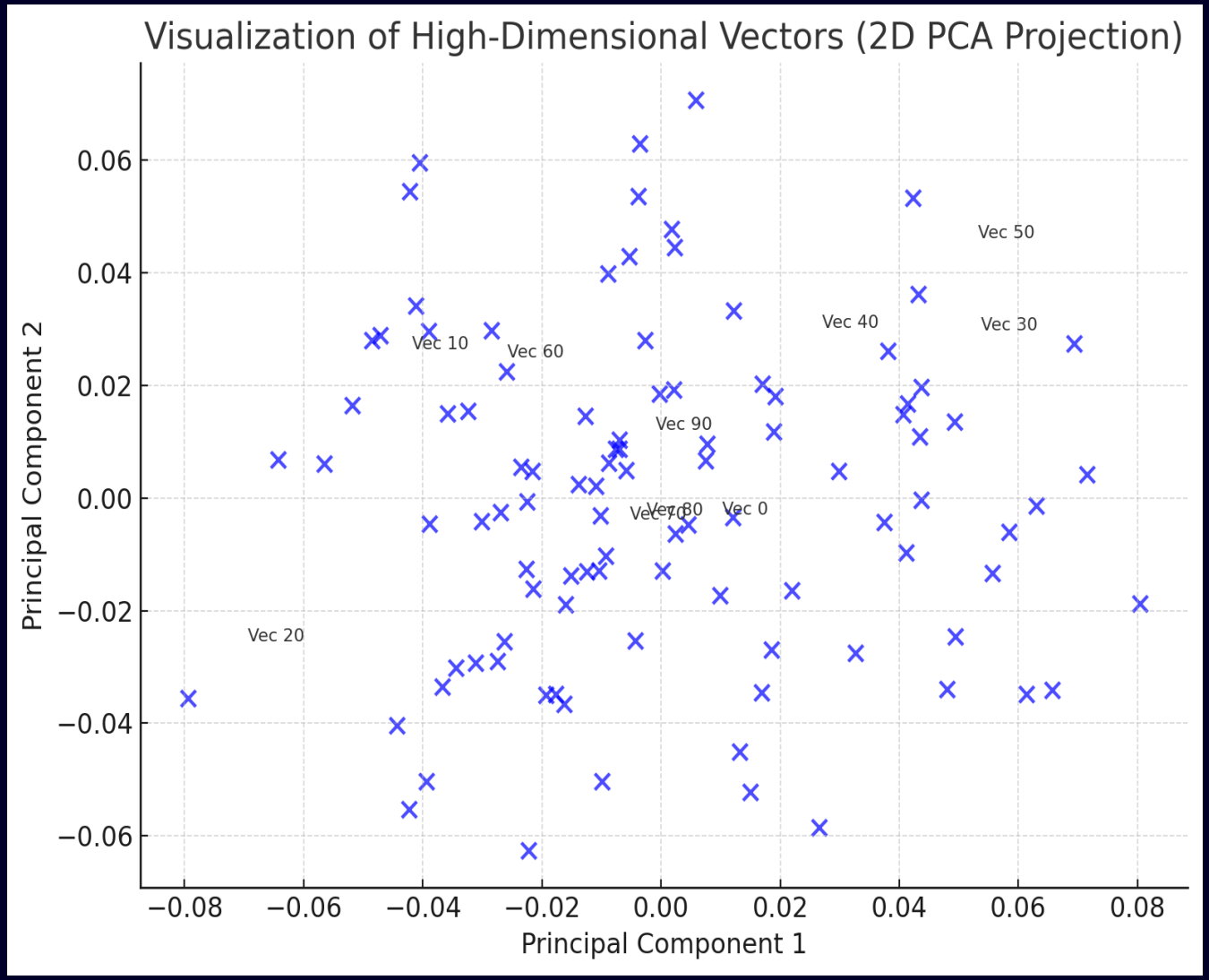
An **embedding** is a learned representation of objects (e.g., words, images, or entities) as vectors in a continuous vector space.
It captures the **semantic relationships** between objects by placing similar objects closer together in the vector space.

•To each chunk we have, we associate it to a high dimensional vector

Key Features
•High-dimensional numerical vectors represent objects.
•Similarity is measured using metrics like **cosine similarity** or **Euclidean distance**.

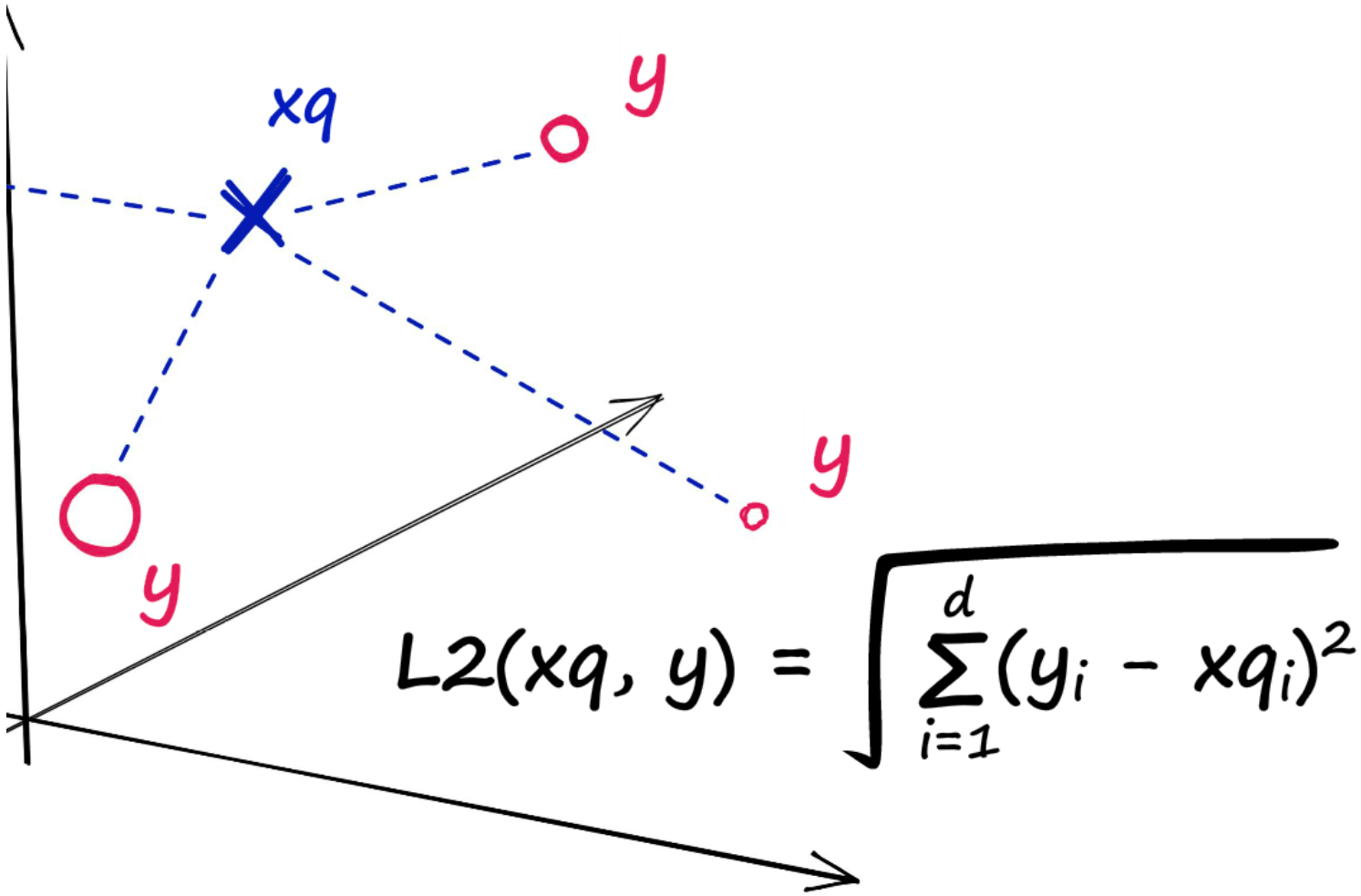
On this application Siemens Rest Apis are used to create embeddings, Provided by :

SIEMENS Internal Developer Portal



Restricted | © Ergi Toshkezi

Faiss



Finding Closest Vectors with Faiss:

• **Process**:

• Faiss is used to perform similarity searches on vector embeddings derived from a knowledge base.

•**Output**: The top **k most relevant vectors** are identified and ordered by their distance to the query (from smallest to largest distance)

IndexFlatL2 method is used:

- IndexFlatL2 measures the L2 (or Euclidean) distance between all given points between our query vector, and the vectors loaded into the index. It's simple, very accurate, but not too fast.

- L2 distance calculation between a query vector xq and our indexed vectors (shown as y)

Restricted | © Ergi Toshkezi

LLM

- Preparing the Payload for Siemens REST API:

•The top **k closest vectors**, along with their associated text and distance values, are formatted into a payload.

•The payload is passed to the Siemens REST API to enable contextual processing.

- Role of the LLM in Summarization:

•The **Siemens REST API LLM** processes the relevant information from the payload.

•It **summarizes** the text data while:

- •Prioritizing content based on relevance to the user query.
- •Taking the **distance value** into account to weigh importance.

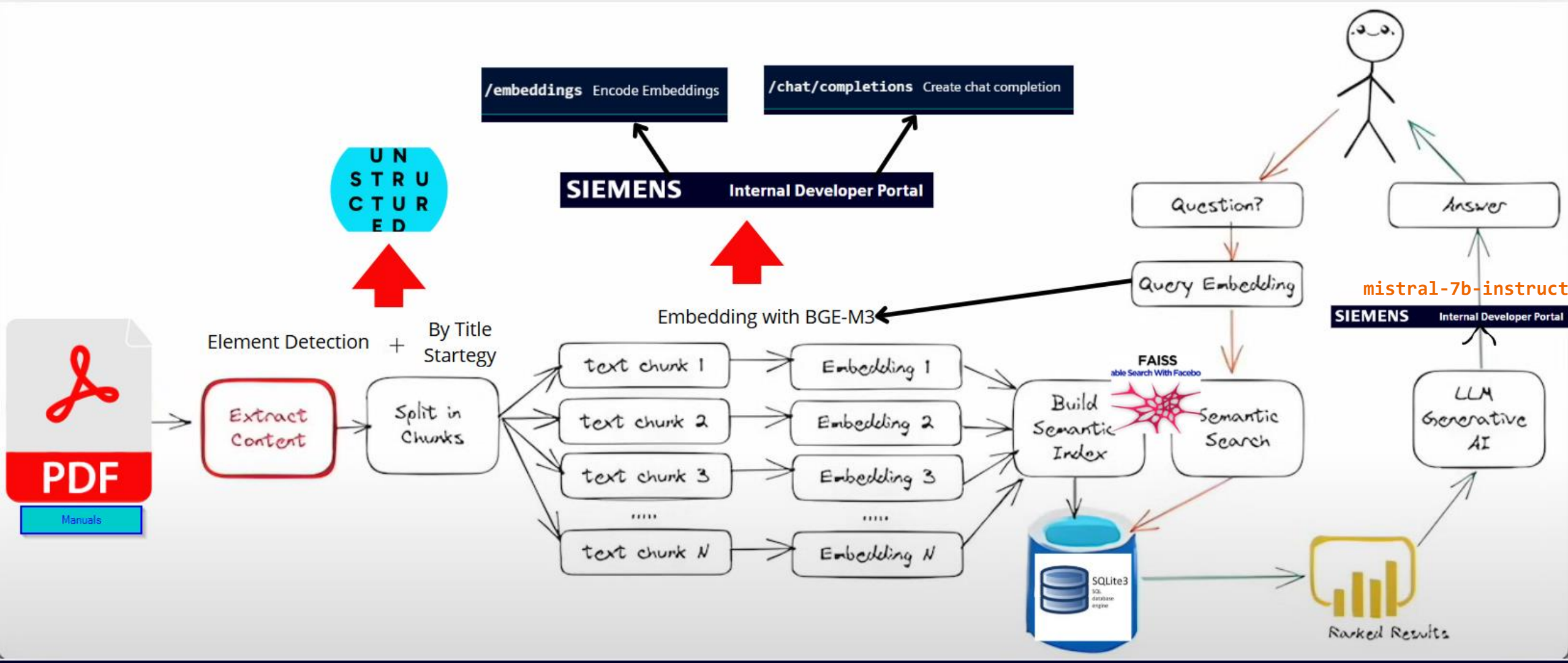
```
# Format FAISS results as a prompt for the LLM API
results_text = "\n".join(
    [f"Result {i + 1}: {result['text']} (distance: {result['distance']:.4f})" for i, result in enumerate(faiss_results)]
)

# Construct the prompt for the LLM
prompt = (
    f"User question: {query}\n\n"
    f"Here are top related results:\n{results_text}\n\n"
    "Please summarize or reorder the results based on their relevance to the question."
)

# Send the structured prompt to the LLM API
```


Restricted | © Ergi Toshkezi

How It Works



Restricted | © Ergi Toshkezi

Thank You!

Restricted | © Ergi Toshkezi